

Rapport Technique

Base de Données Distribuée

Projet E-Shop Multi-Sites

Abdelhamid SAIDI & Anas RIFAK

9 juin 2026

Résumé

Ce rapport présente une implémentation complète d'une architecture de base de données distribuée pour un système d'e-commerce multi-sites. Le projet démontre les concepts clés de la distribution horizontale des données, la synchronisation automatique via déclencheurs (triggers) et la gestion des bases de données liées (database links) sous Oracle Database. Une attention particulière est portée aux stratégies de fragmentation, aux mécanismes de réplication et aux considérations de performance en environnement distribué.

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs du Projet	4
1.3	Portée	4
2	Architecture Système	5
2.1	Vue d'ensemble Architecturale	5
2.1.1	Composants Principaux	5
2.2	Schéma de Données	5
2.2.1	Categories	5
2.2.2	Produits	5
2.2.3	Clients	5
2.2.4	Employes	6
2.2.5	Commandes	6
2.2.6	LigneCommandes	6
2.3	Stratégies de Fragmentation	6
2.3.1	Fragmentation Horizontale Partielle (Scénario 1)	6
2.3.2	Scénario 2 : Fragmentation Métier Avancée	6
2.4	Relations entre Entités	6

3	Implémentation Technique	7
3.1	Environnement de Déploiement	7
3.1.1	Conteneurisation Docker	7
3.1.2	Cycle de Démarrage	7
3.2	Gestion des Bases de Données Liées	8
3.2.1	Création des Liens	8
3.2.2	Utilisation des Liens	8
3.3	Mécanismes de Synchronisation par Déclencheurs	9
3.3.1	Architecture des Déclencheurs	9
3.3.2	Déclencheur d'Insertion (SYC_INSERT_LIGNE)	9
3.3.3	Déclencheur de Mise à Jour (SYC_UPDATE_LIGNE)	10
3.3.4	Déclencheur de Suppression (SYC_DELETE_LIGNE)	10
3.4	Procédures de Synchronisation Distante	11
4	Considérations de Performance	12
4.1	Latence Réseau	12
4.1.1	Impact sur les Requêtes Distribuées	12
4.1.2	Stratégies d'Optimisation	12
4.2	Cohérence des Données	12
4.2.1	Garanties de Cohérence	12
4.2.2	Scénarios de Défaillance	12
4.3	Scalabilité	12
4.3.1	Capacité de Données	12
4.3.2	Gestion de la Croissance	13
4.4	Métriques de Performance	13
4.4.1	Requêtes de Diagnostic	13
5	Tests et Validation	14
5.1	Suite de Tests Automatisée	14
5.2	Cas de Test Clés	14
5.2.1	Test d'Insertion Basique	14
5.2.2	Test de Mise à Jour avec Transition	14
5.2.3	Test de Suppression	15
5.3	Exécution des Tests	15
6	Gestion Opérationnelle	16
6.1	Déploiement	16
6.1.1	Pré-requis Système	16
6.1.2	Procédure de Déploiement	16
6.2	Monitoring et Diagnostique	16
6.2.1	Vérification de la Santé	16
6.2.2	Résolution de Problèmes Courants	17
6.3	Maintenance	17
6.3.1	Arrêt Contrôlé	17
6.3.2	Sauvegardes	17

7	Extensions et Améliorations Futures	18
7.1	Court Terme	18
7.2	Moyen Terme	18
7.3	Long Terme	18
8	Conclusion	19
8.1	Réalisations	19
8.2	Apprentissages	19
8.3	Applicabilité	19
8.4	Perspectives	19
A	Annexes	20
A.1	A. Syntaxes SQL Référence	20
	A.1.1 Connexion aux Bases	20
	A.1.2 Requêtes Distribuées Fréquentes	20
A.2	B. Configuration des Paramètres	21
	A.2.1 Variables d'Environnement	21
	A.2.2 Paramètres Modifiables	21
A.3	C. Glossaire Technique	21

1 Introduction

1.1 Contexte

Les systèmes d'information modernes doivent souvent gérer des données distribuées géographiquement. Un e-commerce multisite doit maintenir la cohérence des données tout en distribuant intelligemment les informations selon la localisation géographique, les besoins métier et les considérations de performance.

1.2 Objectifs du Projet

Ce projet vise à :

- Implémenter une architecture de base de données distribuée avec Oracle Database
- Démontrer la fragmentation horizontale des données
- Implémenter la synchronisation automatique des données via déclencheurs
- Valider l'intégrité des données dans un environnement multi-sites
- Analyser les performances des requêtes distribuées
- Fournir une base de données reproductible via conteneurisation Docker

1.3 Portée

Ce projet couvre :

- Architecture logique avec une base globale et deux sites
- Implémentation de deux scénarios de fragmentation distincts
- Schéma de base de données complet (7 tables principales)
- Mécanismes de synchronisation basés sur les déclencheurs
- Suite de tests automatisée
- Environnement conteneurisé reproductible

2 Architecture Système

2.1 Vue d'ensemble Architecturale

L'architecture proposée repose sur un modèle hybride combinant :

- Une base de données **globale centralisée** qui orchestre la distribution
- Deux bases **sites distribués** hébergeant des sous-ensembles de données
- Des mécanismes de synchronisation par **déclencheurs et bases de données liées**

2.1.1 Composants Principaux

Base de Données Globale La base globale :

- Contient une réplique complète des données
- Exécute les déclencheurs de synchronisation
- Maintient les connexions de bases de données liées vers les sites
- Sert de point d'entrée unique pour les écritures

Bases de Sites Chaque site :

- Héberge un sous-ensemble des données selon les critères de fragmentation
- Peut servir les requêtes locales sans latence réseau
- Demeure synchronisé via les déclencheurs de la base globale

Bases de Données Liées Oracle Database Links permettent :

- L'accès transparent aux données distantes
- L'exécution de procédures stockées à distance
- L'utilisation de syntaxe SQL standard avec l'extension `@link_name`

2.2 Schéma de Données

Le schéma implémente un domaine métier d'e-commerce avec les entités suivantes :

2.2.1 Categories

- **idcateg** (PK) : Identifiant unique de la catégorie
- **nomcateg** : Nom descriptif de la catégorie

2.2.2 Produits

- **idproduit** (PK) : Identifiant unique du produit
- **idcateg** (FK) : Référence vers Categories
- **designation** : Description du produit
- **prixunitaire** : Prix unitaire en euros

2.2.3 Clients

- **idclient** (PK) : Identifiant unique du client
- **codeclient** : Code métier du client
- **societe** : Nom de la société
- **contact** : Personne de contact
- **adresse, ville, pays** : Informations de localisation

2.2.4 Employes

- **idemploye** (PK) : Identifiant unique de l'employé
- **nom, prenom** : Identité de l'employé
- **fonction** : Rôle dans l'organisation

2.2.5 Commandes

- **idcommande** (PK) : Identifiant unique de la commande
- **idclient** (FK) : Client passeur de la commande
- **idemploye** (FK) : Employé responsable
- **datecommande** : Date/heure de création

2.2.6 LigneCommandes

- **idlignecommande** (PK) : Identifiant unique de la ligne
- **idcommande** (FK) : Référence vers Commandes
- **idproduit** (FK) : Produit commandé
- **quantite** : Quantité commandée
- **remise** : Pourcentage de remise appliqué

2.3 Stratégies de Fragmentation

2.3.1 Fragmentation Horizontale Partielle (Scénario 1)

Le scénario 1 implémente une fragmentation horizontale partielle basée sur les catégories de produits :

Site	Critère	Description
Site1	<code>idcateg = 50 AND quantite > 100</code>	Lignes de commande pour la catégorie 50 avec quantité élevée
Site2	<code>idcateg = 35 AND quantite > 50</code>	Lignes de commande pour la catégorie 35 avec quantité modérée
Global	Tous les enregistrements non distribués	Réplique complète + données non fragmentées

TABLE 1 – Fragmentation du scénario 1

Cette approche crée un ensemble de données :

- **Fragmenté** : Les lignes respectant les critères sont distribuées
- **Non fragmenté** : Les autres lignes restent uniquement dans la base globale

2.3.2 Scénario 2 : Fragmentation Métier Avancée

Le scénario 2 applique des règles métier plus sophistiquées permettant :

- Fragmentation multi-critères
- Règles basées sur plusieurs attributs
- Logique métier complexe dans les déclencheurs

2.4 Relations entre Entités

Relation	Type	Détails
Produits.idcateg → Categories.idcateg	FK	N :1
LigneCommandes.idcommande → Commandes.idcommande	FK	N :1
LigneCommandes.idproduit → Produits.idproduit	FK	N :1
Commandes.idclient → Clients.idclient	FK	N :1
Commandes.idemploye → Employes.idemploye	FK	0..1 :1

TABLE 2 – Contraintes de clés étrangères

3 Implémentation Technique

3.1 Environnement de Déploiement

3.1.1 Conteneurisation Docker

Le projet utilise Docker Compose pour orchestrer trois conteneurs Oracle Database :

Listing 1 – Structure de compose.yml

```

1 services:
2   site1_db:
3     build: ./site1_db
4     ports: ["1522:1521"]
5     healthcheck: 30s interval, 20 retries
6
7   site2_db:
8     build: ./site2_db
9     ports: ["1523:1521"]
10    healthcheck: 30s interval, 20 retries
11
12   global_db:
13     build: ./global_db
14     depends_on: [site1_db, site2_db]
15     ports: ["1521:1521"]
16     healthcheck: 30s interval, 20 retries

```

3.1.2 Cycle de Démarrage

Le cycle de démarrage orchestré suit ces étapes :

1. **Construction** : Compilation des images Docker pour les trois bases
2. **Initialisation Site1** : Création du schéma et des utilisateurs
3. **Initialisation Site2** : Création du schéma et des utilisateurs
4. **Vérification** : Health checks confirment la disponibilité des sites
5. **Initialisation Global** : Création du schéma globale
6. **Création des Liens** : Établissement des database links
7. **Sélection du Scénario** : Application des déclencheurs appropriés
8. **Chargement des Données** : Insertion des données de test

Durée totale : 5 à 10 minutes selon la configuration matérielle.

3.2 Gestion des Bases de Données Liées

3.2.1 Création des Liens

Les database links sont créés dans la base globale :

Listing 2 – Création des liens vers les sites

```
1 GRANT CREATE DATABASE LINK TO eshop;
2 CONNECT eshop/eshop123@localhost:1521/FREEPDB1
3
4 CREATE DATABASE LINK site1_link
5     CONNECT TO site1 IDENTIFIED BY site1123
6     USING '(DESCRIPTION=
7         (ADDRESS=(PROTOCOL=TCP)
8         (HOST=eshop_site1_db)(PORT=1521))
9         (CONNECT_DATA=(SERVICE_NAME=FREEPDB1)))';
10
11 CREATE DATABASE LINK site2_link
12     CONNECT TO site2 IDENTIFIED BY site2123
13     USING '(DESCRIPTION=
14         (ADDRESS=(PROTOCOL=TCP)
15         (HOST=eshop_site2_db)(PORT=1521))
16         (CONNECT_DATA=(SERVICE_NAME=FREEPDB1)))';
```

3.2.2 Utilisation des Liens

Les liens permettent la syntaxe transparente :

Listing 3 – Requêtes via database links

```
1 -- Requête locale
2 SELECT COUNT(*) FROM LigneCommandes;
3
4 -- Requête distante sur Site1
5 SELECT COUNT(*) FROM LigneCommandes@site1_link;
6
7 -- Requête distante sur Site2
8 SELECT COUNT(*) FROM LigneCommandes@site2_link;
9
10 -- Join entre local et distant
11 SELECT l.idlignecommande, l.quantite, p.designation
12 FROM LigneCommandes l
13 INNER JOIN Produits p ON l.idproduit = p.idproduit
14 WHERE l.quantite > 50;
15
16 SELECT l.idlignecommande, l.quantite
17 FROM LigneCommandes@site1_link l
18 WHERE l.quantite > 100;
```


3.3 Mécanismes de Synchronisation par Déclencheurs

3.3.1 Architecture des Déclencheurs

Trois catégories de déclencheurs sont implémentées :

Type	Nombre	Fonction
INSERT	3	Synchronisation lors d'insertion
UPDATE	3	Gestion des transitions entre fragments
DELETE	3	Suppression des données distantes

TABLE 3 – Classification des déclencheurs

3.3.2 Déclencheur d'Insertion (SYC_INSERT_LIGNE)

Le déclencheur d'insertion exécute la logique suivante :

1. **Récupération des données** : Lecture des colonnes de la nouvelle ligne
2. **Evaluation des critères** : Vérification des conditions de fragmentation
3. **Fusion des données** : Récupération des données associées (client, produit, etc.)
4. **Insertion conditionnelle** : Appel à des procédures pour chaque site matching

Structure pseudo-code :

Listing 4 – Logique du déclencheur INSERT simplifié

```

1  TRIGGER SYC_INSERT_LIGNE
2  AFTER INSERT ON LigneCommandes
3  FOR EACH ROW
4  DECLARE
5      v_idcateg NUMBER;
6      v_quantite NUMBER;
7  BEGIN
8      -- Récupérer l'ID catégorie du produit
9      SELECT idcateg INTO v_idcateg
10     FROM Produits WHERE idproduit = :NEW.idproduit;
11
12     -- Scénario 1 : Fragmentation par catégorie
13     IF SCENARIO = 1 THEN
14         -- Critère Site1
15         IF v_idcateg = 50 AND :NEW.quantite > 100 THEN
16             push_to_site1(:NEW.idlignecommande, ...);
17         END IF;
18
19         -- Critère Site2
20         IF v_idcateg = 35 AND :NEW.quantite > 50 THEN
21             push_to_site2(:NEW.idlignecommande, ...);
22         END IF;
23     END IF;
24
25     EXCEPTION WHEN OTHERS THEN
26         -- Gestion d'erreur
27     END;

```

3.3.3 Déclencheur de Mise à Jour (SYC_UPDATE_LIGNE)

Le déclencheur UPDATE gère les transitions d'état :

- **Pas de changement** : Mise à jour simple sur les sites où la ligne existe
- **Entrée dans un fragment** : Insertion du côté distant
- **Sortie d'un fragment** : Suppression du côté distant
- **Migration** : Suppression puis insertion sur un autre site

Logique :

Listing 5 – Logique du déclencheur UPDATE simplifié

```

1  TRIGGER SYC_UPDATE_LIGNE
2  AFTER UPDATE ON LigneCommandes
3  FOR EACH ROW
4  DECLARE
5      v_old_categ  NUMBER;
6      v_new_categ  NUMBER;
7  BEGIN
8      -- R cup rer cat gories ancienne et nouvelle
9      SELECT idcateg INTO v_old_categ
10     FROM Produits WHERE idproduit = :OLD.idproduit;
11     SELECT idcateg INTO v_new_categ
12     FROM Produits WHERE idproduit = :NEW.idproduit;
13
14     -- V rifier transitions
15     IF v_old_categ = 50 AND v_new_categ != 50 THEN
16         -- Sortie de Site1 : supprimer
17         DELETE FROM LigneCommandes@site1_link
18         WHERE idlignecommande = :NEW.idlignecommande;
19     ELSIF v_old_categ != 50 AND v_new_categ = 50 THEN
20         -- Entr e en Site1 : ins rer
21         push_to_site1(:NEW.idlignecommande, ...);
22     END IF;
23
24 END;
```

3.3.4 Déclencheur de Suppression (SYC_DELETE_LIGNE)

Le déclencheur DELETE supprime les données distantes :

Listing 6 – Logique du déclencheur DELETE

```

1  TRIGGER SYC_DELETE_LIGNE
2  AFTER DELETE ON LigneCommandes
3  FOR EACH ROW
4  BEGIN
5      -- Supprimer du Site1 s'il existe
6      BEGIN
7          DELETE FROM LigneCommandes@site1_link
8          WHERE idlignecommande = :OLD.idlignecommande;
9          COMMIT;
10     EXCEPTION WHEN OTHERS THEN NULL; -- Ignore si absent
11 END;
```

```
12
13 -- Supprimer du Site2 s'il existe
14 BEGIN
15     DELETE FROM LigneCommandes@site2_link
16     WHERE idlignecommande = :OLD.idlignecommande;
17     COMMIT;
18     EXCEPTION WHEN OTHERS THEN NULL; -- Ignore si absent
19     END;
20 END;
```

3.4 Procédures de Synchronisation Distante

Les procédures `push_to_site1` et `push_to_site2` encapsulent la logique d'insertion distante :

Listing 7 – Procédure de push vers Site1

```
1 PROCEDURE push_to_site1(
2     p_idlignecommande IN NUMBER,
3     p_idcommande      IN NUMBER,
4     ...
5 ) IS
6 BEGIN
7     INSERT INTO LigneCommandes@site1_link
8     (idlignecommande, idcommande, idproduit, quantite, remise)
9     VALUES
10    (p_idlignecommande, p_idcommande, p_idproduit,
11     p_quantite, p_remise);
12
13    COMMIT;
14
15 EXCEPTION
16    WHEN DUP_VAL_ON_INDEX THEN
17        UPDATE LigneCommandes@site1_link
18        SET quantite = p_quantite, remise = p_remise
19        WHERE idlignecommande = p_idlignecommande;
20        COMMIT;
21
22    WHEN OTHERS THEN
23        DBMS_OUTPUT.PUT_LINE('Erreur Site1: ' || SQLERRM);
24        RAISE;
25 END push_to_site1;
```

4 Considérations de Performance

4.1 Latence Réseau

4.1.1 Impact sur les Requêtes Distribuées

Les requêtes qui accèdent aux données distantes via database links subissent :

- **Latence réseau** : Round-trip time (RTT) pour chaque opération
- **Overhead de protocole** : Encapsulation TNS (Transparent Network Substrate)
- **Contexte de transaction** : Maintenance de l'état transactionnel distribué

Formule de coût estimé :

$$T_{total} = T_{execution_locale} + n \times (RTT + T_{execution_distante})$$

Où n est le nombre d'aller-retours réseau.

4.1.2 Stratégies d'Optimisation

- **Batching** : Regrouper les opérations pour réduire n
- **Vues matérialisées** : Cacher les données fréquemment accédées
- **Pushdown de prédicat** : Exécuter la sélection côté distant avant retour

4.2 Cohérence des Données

4.2.1 Garanties de Cohérence

Le design assure :

- **Cohérence immédiate** : Les déclencheurs synchronisent immédiatement
- **Atomicité** : Chaque opération distante se termine avec COMMIT
- **Isolation** : Les transactions distribuées utilisent le READ COMMITTED

4.2.2 Scénarios de Défaillance

Scénario	Symptôme	Mitigation
Site distante indisponible	INSERT échoue	Retry avec backoff exponentiel
Lien corrompu	TIMED_OUT exception	Recréer le lien
Données orphelines	Ligne sur Site mais pas Global	Reconciliation manuelle

TABLE 4 – Modes de défaillance et stratégies

4.3 Scalabilité

4.3.1 Capacité de Données

Estimation pour l'implémentation actuelle :

- **Commandes** : 10^6 enregistrements
- **LigneCommandes** : 10^7 enregistrements (10 lignes par commande)
- **Stockage** : ≈ 200 GB par site

4.3.2 Gestion de la Croissance

1. **Partitionnement** : Diviser les tables par plages temporelles
2. **Archivage** : Déplacer les données anciennes vers des tables historiques
3. **Ajout de sites** : Étendre à 3+ sites avec fragmentation multi-voies

4.4 Métriques de Performance

4.4.1 Requêtes de Diagnostic

Listing 8 – Métriques d'exécution

```
1  -- Performance d'une requête
2  EXPLAIN PLAN FOR
3  SELECT * FROM LigneCommandes WHERE quantite > 100;
4
5  SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());
6
7  -- Volume de données par site
8  SELECT COUNT(*) FROM LigneCommandes;
9  SELECT COUNT(*) FROM LigneCommandes@site1_link;
10 SELECT COUNT(*) FROM LigneCommandes@site2_link;
11
12 -- Temps de réponse
13 SET TIMING ON
14 SELECT l.*, p.designation
15 FROM LigneCommandes l
16 JOIN Produits p ON l.idproduit = p.idproduit
17 WHERE l.quantite > 500;
```

5 Tests et Validation

5.1 Suite de Tests Automatisée

La suite de tests vérifie :

1. **Intégrité structurale** : Présence des tables et colonnes
2. **Contraintes** : Clés primaires et étrangères fonctionnelles
3. **Synchronisation** : Données correctement distribuées
4. **Déclaration des déclencheurs** : Trigger actifs et corrects
5. **Cohérence globale** : Somme des sites = global (moins non-fragmenté)

5.2 Cas de Test Clés

5.2.1 Test d'Insertion Basique

Listing 9 – Test d'insertion et vérification

```
1 -- Ins rer une cat gorie
2 INSERT INTO Categories VALUES (50, 'Electronics');
3
4 -- Ins rer un produit
5 INSERT INTO Produits VALUES (5001, 50, 'Laptop', 999.99);
6
7 -- Ins rer un client et un employ
8 INSERT INTO Clients VALUES (1, 'CLI001', 'TechCorp', 'John',
9 '123 Main St', 'Paris', 'France');
10 INSERT INTO Employes VALUES (1, 'Smith', 'Alice', 'Sales Manager'
11 );
12
13 -- Ins rer une commande
14 INSERT INTO Commandes VALUES (1001, 1, 1, SYSDATE);
15
16 -- Ins rer une ligne (quantit > 100, donc sync vers Site1)
17 INSERT INTO LigneCommandes VALUES (1, 1001, 5001, 150, 0);
18 COMMIT;
19
20 -- V rifier la synchronisation
21 SELECT COUNT(*) FROM LigneCommandes; -- 1 ligne globale
22 SELECT COUNT(*) FROM LigneCommandes@site1_link; -- 1 ligne
23 SELECT COUNT(*) FROM LigneCommandes@site2_link; -- 0 lignes
```

5.2.2 Test de Mise à Jour avec Transition

Listing 10 – Test de transition entre fragments

```
1 -- Changer le produit d'une ligne
2 -- (passage de cat gorie 50      cat gorie 35)
3 UPDATE LigneCommandes
4 SET idproduit = 3502 -- Produit de cat gorie 35
5 WHERE idlignecommande = 1;
```

```
6 COMMIT;
7
8 -- Site1 : ligne supprimée (catégorie 35 seul)
9 SELECT COUNT(*) FROM LigneCommandes@site1_link; -- 0 lignes
10
11 -- Site2 : ligne insérée (catégorie 35 + quantité > 50)
12 SELECT COUNT(*) FROM LigneCommandes@site2_link; -- 1 ligne
```

5.2.3 Test de Suppression

Listing 11 – Test de suppression en cascade

```
1 -- Supprimer une ligne
2 DELETE FROM LigneCommandes WHERE idlignecommande = 1;
3 COMMIT;
4
5 -- Vérifier suppression distribuée
6 SELECT COUNT(*) FROM LigneCommandes; -- 0
7 SELECT COUNT(*) FROM LigneCommandes@site1_link; -- 0
8 SELECT COUNT(*) FROM LigneCommandes@site2_link; -- 0
```

5.3 Exécution des Tests

Les tests s'exécutent via la suite automatisée :

Listing 12 – Exécution des tests

```
1 # Exécuter la suite de tests
2 docker exec eshop_global_db sqlplus sys/oracle123@FREEPDB1 \
3   as sysdba @/tests/test_all_sites.sql
4
5 # Exécuter un test spécifique
6 docker exec eshop_global_db sqlplus eshop/eshop123@FREEPDB1 \
7   <<EOF
8 SET SERVEROUTPUT ON
9 DECLARE
10   v_count NUMBER;
11 BEGIN
12   SELECT COUNT(*) INTO v_count FROM LigneCommandes;
13   DBMS_OUTPUT.PUT_LINE('Total lines: ' || v_count);
14 END;
15 /
16 EOF
```

6 Gestion Opérationnelle

6.1 Déploiement

6.1.1 Pré-requis Système

Composant	Minimum
Mémoire RAM	8 GB
Espace disque	40 GB
CPU	4 cœurs
Docker	20.10+
Docker Compose	2.0+

TABLE 5 – Spécifications matérielles requises

6.1.2 Procédure de Déploiement

1. **Récupération du code** : Clone ou extraction du projet
2. **Vérification pré-déploiement** :
 - Docker en cours d'exécution
 - Ports 1521, 1522, 1523 disponibles
 - Espace disque suffisant
3. **Construction et lancement** :

```
1 cd Distributed-DB
2 docker compose up --build
```

4. **Vérification post-déploiement** :

```
1 docker compose ps # Tous les conteneurs HEALTHY
2 docker compose logs global_db | grep "Startup"
```

5. **Tests** : Exécuter la suite de tests complète

Durée : 5-10 minutes

6.2 Monitoring et Diagnostic

6.2.1 Vérification de la Santé

Listing 13 – Commandes de monitoring

```
1 # Statut des conteneurs
2 docker compose ps
3
4 # Logs d'attente
5 docker compose logs -f global_db
6
7 # Vérifier la connectivité
8 docker exec eshop_global_db sqlplus -s -l \
9   eshop/eshop123@FREEPDB1 <<< "SELECT 1 FROM DUAL;"
```



```

10
11 # Lister les bases de données liées
12 docker exec eshop_global_db sqlplus eshop/eshop123@FREEPDB1 \
13 <<< "SELECT db_link, username FROM user_db_links;"

```

6.2.2 Résolution de Problèmes Courants

Problème	Cause Probable	Solution
Les conteneurs crash	OOM (Out of Memory)	Augmenter RAM disponible
Health check échoue	Oracle non prêt	Attendre 180s+
Lien indisponible	Site distant down	Vérifier status avec ps
Tests échouent	Données non sync	Attendre et réessayer
Port déjà utilisé	Service réseau conflictuel	Changer port dans compose.yml

TABLE 6 – Troubleshooting courant

6.3 Maintenance

6.3.1 Arrêt Contrôlé

```

1 # Arr t gracieux
2 docker compose down
3
4 # Suppression compl te (données conserv es dans volumes)
5 docker compose down -v
6
7 # Suppression des images
8 docker compose down --rmi all

```

6.3.2 Sauvegardes

Listing 14 – Procédure de sauvegarde

```

1 # Exporter les données depuis Oracle
2 docker exec eshop_global_db expdp eshop/eshop123 \
3   DIRECTORY=dpump_dir DUMPFILE=eshop_backup.dmp
4
5 # Ou utiliser Docker volumes
6 docker cp eshop_global_db:/opt/oracle/oradata/backup.tar \
7   ./backup_$(date +%Y%m%d).tar

```

7 Extensions et Améliorations Futures

7.1 Court Terme

- **Réplication multidirectionnelle** : Permettre les écritures sur les sites
- **Résolution de conflits** : Implémenter stratégies LWW (Last-Write-Wins)
- **Monitoring avancé** : Dashboard des métriques temps réel

7.2 Moyen Terme

- **Haute disponibilité** : Failover automatique avec Data Guard
- **Chiffrement** : TLS pour les communications inter-sites
- **Vues matérialisées** : Caching pré-calculé des données distribuées

7.3 Long Terme

- **Passage à Kubernetes** : Orchestration multi-cloud
- **NoSQL hybride** : Support de MongoDB ou Cassandra
- **Temps réel** : Event streaming avec Kafka

8 Conclusion

Ce projet démontre avec succès une implémentation complète d'une architecture de base de données distribuée utilisant Oracle Database. Les points clés de cette réalisation sont :

8.1 Réalisations

1. **Architecture** : Design robuste combinant base globale et sites distribués
2. **Synchronisation** : Mécanismes fiables basés sur les déclencheurs
3. **Flexibilité** : Support de multiples scénarios de fragmentation
4. **Reproductibilité** : Environnement entièrement conteneurisé
5. **Validation** : Suite de tests complète couvrant les cas nominaux et limites

8.2 Apprentissages

Le projet illustre les concepts fondamentaux de la distribution de données :

- **Fragmentation horizontale** : Partitionnement logique des données par critères métier
- **Transparence** : Database links masquent la complexité de l'accès distribué
- **Trade-offs** : Compromise entre performance locale et cohérence globale
- **Opérationnel** : Défis de monitoring et maintenance en environnement distribué

8.3 Applicabilité

Cette approche convient pour :

- Systèmes d'e-commerce multi-régions
- Architectures avec localisation des données
- Cas d'usage nécessitant autonomie locale et cohérence globale

Elle est moins adaptée pour :

- Systèmes requérant des transactions distribuées forte cohérence
- Données hautement volatiles avec beaucoup de migrations inter-sites
- Cas critique latence très basse

8.4 Perspectives

L'évolution naturelle consiste à :

1. Ajouter réplication bidirectionnelle
2. Implémenter haute disponibilité et failover
3. Étendre à architectures cloud-native
4. Intégrer monitoring et observabilité avancée

Ce projet constitue une base solide pour explorer les systèmes distribués en pratique.

A Annexes

A.1 A. Syntaxes SQL Référence

A.1.1 Connexion aux Bases

Listing 15 – Commandes de connexion

```
1 # Global
2 sqlplus eshop/eshop123@localhost:1521/FREEPDB1
3
4 # Site1
5 sqlplus site1/site1123@localhost:1522/FREEPDB1
6
7 # Site2
8 sqlplus site2/site2123@localhost:1523/FREEPDB1
9
10 # Systeme (Docker)
11 docker exec -it eshop_global_db sqlplus \
12     sys/oracle123@FREEPDB1 as sysdba
```

A.1.2 Requêtes Distribuées Fréquentes

Listing 16 – Requêtes distribuées utiles

```
1 -- Compter les lignes locales
2 SELECT 'Global' AS loc, COUNT(*) cnt
3 FROM LigneCommandes
4 UNION ALL
5 SELECT 'Site1', COUNT(*) FROM LigneCommandes@site1_link
6 UNION ALL
7 SELECT 'Site2', COUNT(*) FROM LigneCommandes@site2_link
8 ORDER BY loc;
9
10 -- Produits non distribués (ni Site1 ni Site2)
11 SELECT COUNT(*) FROM LigneCommandes l
12 WHERE NOT EXISTS (SELECT 1 FROM LigneCommandes@site1_link
13                   WHERE idlignecommande = l.idlignecommande)
14    AND NOT EXISTS (SELECT 1 FROM LigneCommandes@site2_link
15                   WHERE idlignecommande = l.idlignecommande);
16
17 -- Trouver les doublons potentiels
18 SELECT idlignecommande, COUNT(*) cnt
19 FROM (
20     SELECT idlignecommande FROM LigneCommandes
21     UNION ALL
22     SELECT idlignecommande FROM LigneCommandes@site1_link
23     UNION ALL
24     SELECT idlignecommande FROM LigneCommandes@site2_link
25 )
26 GROUP BY idlignecommande
27 HAVING COUNT(*) > 1;
```

A.2 B. Configuration des Paramètres

A.2.1 Variables d'Environnement

Variable	Valeur	Usage
SCENARIO	1 ou 2	Sélectionne les déclencheurs
ORACLE_PASSWORD	oracle123	Password SYS
ORACLE_CHARACTERSET	AL32UTF8	Charset (UTF-8)

TABLE 7 – Variables d'environnement Docker

A.2.2 Paramètres Modifiables

Listing 17 – Paramètres dans compose.yml

```
1 # Modifier le port externe de la base globale
2 ports:
3   - "1521:1521" # [externe]:[interne]
4
5 # Augmenter la mmoire allouée
6 environment:
7   ORACLE_MEMORY: "2G" # Par défaut: 1G
8
9 # Changer le mot de passe
10 environment:
11   ORACLE_PASSWORD: "newpassword123"
```

A.3 C. Glossaire Technique

Terme	Définition
Database Link	Mécanisme Oracle permettant l'accès transparent à une base distante
Déclencheur	Procédure stockée exécutée automatiquement lors d'événements DDL/DML
Fragmentation	Partitionnement logique des données selon critères métier
Horizontal	Fragmentation par lignes (vs vertical par colonnes)
Synchronisation	Propagation automatique des changements vers sites distribués
Cohérence	Garantie que les données restent dans un état valide
Réplica	Copie d'une donnée sur plusieurs sites
Push	Envoi proactif des données du serveur central vers sites
Pull	Récupération des données depuis sites vers serveur central

TABLE 8 – Glossaire